

ShiftFlow **mvp-0.1**

Arquitectura y desarrollo asistido por IA

Dos entregables: un producto demostrable · un método reutilizable (SDAF)

Ventana: 1–22 ago 2026 · ~96 h de capacidad · congelado el **2026-08-17**

El recorrido funcional está en las **slides de producto**

Qué se somete a evaluación

Entregable	Qué es	Dónde se verifica
Producto	Planificación de turnos con validación de reglas	Etiqueta <code>mvp-0.1</code> + <code>docs/runbook-local.md</code>
Método	SDAF: las especificaciones gobiernan a los agentes	<code>handbook/</code> + <code>specs/</code> + <code>worklogs/</code>

Inventario de evidencia a fecha de la etiqueta

Artefacto	Cantidad
Capítulos de handbook (constitución + anexos)	21 + 4
Especificaciones aprobadas (3 producto · 7 dominio · 5 aplicación · 5 aceptación)	20
Decisiones de arquitectura fechadas (ADR-001 ... ADR-007)	7
Elementos de backlog trazados a spec	15
Registros de trabajo por iteración (25 líneas de trabajo)	61
Contratos de agente (5 activos · 8 stub)	13

La primera decisión fue de alcance

El conocimiento de partida (`knowledge/raw/Domain-Specs-V1.docx`) traía reglas de nivel experto: turnos **pares/impares**, **bolsa mensual** de horas, **cuotas nocturnas**, validación entre meses, *fairness scoring*.

Con un asistente que produce código rápido, el cuello de botella deja de ser escribir: pasa a ser **decidir**. Y la decisión hay que documentarla.

Opción	Por qué se rechazó
Implementar el dominio experto completo	Sistema grande, frágil y no demostrable en ~96 h
Recortar sin registrar	La frontera viviría en la cabeza de una persona; el asistente la cruzaría
Recortar y dejarlo por escrito	Elegida: el techo es citable y frena al agente en Gate 0

El conocimiento no implementado **no se descarta**: queda etiquetado *diferido* en las specs de dominio.

El corte, con identificadores

In — 14 capacidades

C-ORG C-DEP C-EMP C-STT organizaciones, departamentos, empleados, tipos de turno
C-CAL C-ASN calendario mensual y **asignación manual**
C-RUL C-LEA validación de reglas duras y ausencias
C-AUTH C-API C-WEB C-LOC C-OBS acceso, API, cliente único, runtime local, logging
C-PRE presentación de cierre

Out — declarado, no omitido

MAUI · Redis · SignalR · optimización automática · **IA que escribe cuadrantes** · nube como único camino de demo · reglas avanzadas del DOCX

Techo explícito: un bounded context · asignación manual · **máximo 3 reglas duras**

La arquitectura se define por fronteras

#	Frontera	Decisión	Registro
1	Superficie de usuario	Una sola (Blazor Web App); MAUI Blazor Hybrid diferido	ADR-002
2	Lógica de negocio	Solo en Domain ; la UI no reimplementa reglas	ADR-004
3	Contexto delimitado	Uno: <code>WorkforceScheduling</code> ; identidad es subdominio de soporte	ADR-003
4	Ejecución	Local autocontenida ; la nube no es peaje para evaluar	ADR-001

Riesgo específico del trabajo asistido por IA: el agente resuelve el problema donde le resulta cómodo —normalmente en la pantalla— y **erosiona el modelo** sin que nadie lo note. La frontera 2 existe para eso.

Frontera 2, hecha ejecutable

Web

→

ServiceDefaults (+ HttpClient → Api)

Api

→

Application, Infrastructure, ServiceDefaults

Infrastructure

→

Application, Domain

Application

→

Domain

Domain

→

(ninguna referencia a ShiftFlow)

AppHost

→

Api, Web

Proyecto	Rol
ShiftFlow.Domain	Modelo, invariantes y puertos
ShiftFlow.Application	Vertical slices CQRS (MediatR): separa lo que muta de lo que consulta
ShiftFlow.Infrastructure	EF Core / PostgreSQL / adaptadores
ShiftFlow.Api · ShiftFlow.Web	Hosts separados: composition root vs. UI

El grafo no es documentación: **cambiarlo obliga a enmendar el ADR.**

Ejecución local, y decisiones que se enmiendan

Restricción	Pieza que la resuelve
Evaluar sin cuenta en la nube	.NET Aspire: un comando levanta Postgres + Api + Web
Persistencia relacional con invariantes	PostgreSQL + EF Core
Contenedor local ligero, licencia cero	Docker (Compose documentado como contingencia)
Histórico de esquema reproducible	Migraciones EF (ADR-007)

Dos enmiendas registradas, no ocultas

Cambio	Motivo
net10.0 → net9.0 → net10.0	SDK 10 no disponible el 07-ago; revertido al instalarlo
Testcontainers → SQLite + EnsureCreated en integración	Coste de tooling desproporcionado para el corte (ADR-007)

Un ADR con historial vale más que un ADR que parece que acertó a la primera.

Dónde ponemos la IA del producto

La propuesta inicial eran **cinco motores**. Para este corte, dos.

Motor propuesto	Decisión MVP
Scheduling Engine	Dentro: ciclo de vida de turnos, asignación manual
Rule Engine v1	Dentro: mecanismo del BC, no microservicio
Compliance Engine	Absorbido en Rule Engine v1
Optimization Engine	No se implementa
AI Recommendation Engine	No como motor de escritura: solo stub de infraestructura que <i>explica</i>

Alternativa evaluada	Motivo de rechazo
IA que genera y persiste cuadrantes	Viola el principio de producto: IA asistente, confirmación humana
Motor de reglas externo (Drools...)	Exceso de infraestructura para ≤ 3 reglas

Rule Engine v1: la autoridad, con nombre

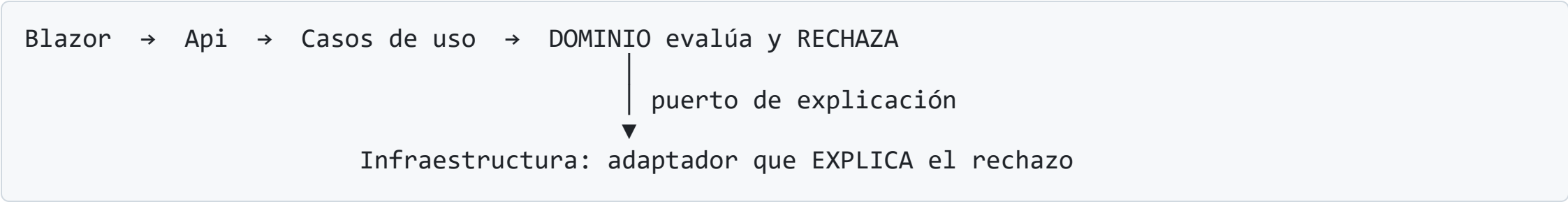
```
Evaluate(candidate: ShiftAssignment, contexto de plan) → ok | RuleViolation[]
```

`AssignShift` **debe** invocar la evaluación **antes de persistir**. No es una recomendación: es criterio de aceptación.

ID	Regla	Semántica precisa
HR-01	No solape	Intervalos semiabiertos <code>[StartAt, EndAt)</code> ; solo estado <code>Assigned</code> ; comparación por <code>EmployeeId</code> . Dos turnos que se tocan en el borde no solapan
HR-02	Ausencia activa	Un <code>Leave</code> activo que cubra cualquier instante del intervalo candidato bloquea
HR-03	Descanso mínimo	Umbral configurable por <code>organization</code> entre turnos adyacentes

Varias violaciones pueden devolverse juntas; **una basta para rechazar**.

La IA explica; el dominio decide



Consecuencia arquitectónica	Por qué importa
La autoridad sobre <i>qué es válido</i> permanece en el dominio	Auditable y cubierto por tests
La IA queda en el borde , tras un puerto	Mañana puede haber un LLM real detrás sin tocar el núcleo
Ninguna regla laboral depende de una respuesta generativa	El sistema no falla si el modelo se equivoca

Asimetría deliberada. Es el patrón que defiendo para sistemas con reglas normativas.

SDAF — jerarquía normativa

```
Conocimiento experto (inmutable, versionado en knowledge/)
└─ Handbook (constitución: 21 capítulos)
    └─ Especificaciones aprobadas — producto · dominio · aplicación · aceptación
        └─ Decisiones de arquitectura fechadas (ADR)
            └─ Backlog (PBI trazado a spec)
                └─ Implementación || tests derivados de la spec
                    └─ Revisión y puertas de control
                        └─ Release + presentación
```

Cada nivel **solo** puede apoyarse en los de arriba. Un test no nace de la implementación: nace de la especificación de aceptación.

La especificación gobierna al agente. No al revés.

Agentes con rol acotado

Agente	Produce	No puede
Specification	Specs desde el conocimiento	Aprobar su propia spec
Architecture	ADR con alternativas y consecuencias	Decidir alcance de producto
Domain + Application	Modelo, invariantes, casos de uso	Tocar la UI
Frontend	Interfaz sobre contratos existentes	Reimplementar una regla
Testing + Review	Pruebas y dictamen de integración	Aprobar su propio código

Cada uno tiene **contrato escrito** (`agents/`) y **prompt versionado** (`prompts/agents/`). Ocho agentes más existen como *stub* para desacoplar post-MVP sin rehacer el router.

Handoff: `Specification → Architecture → Domain+Application → Frontend → Testing+Review`

El relevo va por **registro de trabajo**, no por conversación: si mañana se pierde el historial de chat, el proyecto sigue siendo reconstruible.

Cuatro puertas hacen exigible el método

Puerta	Exige	Si falta algo
0 — antes de codificar	Spec Approved · criterios de aceptación · ADR si el cambio toca fronteras · PBI · worklog abierto	STOP. No se codifica «un poco para ver qué sale»
1 — ejecución	Rebanada vertical · no ampliar lo declarado Out	Corrección en la iteración
2 — integración	Aceptación en verde · revisión con checklist · arranque local sigue funcionando	No se integra
3 — cierre MVP	Demo reproducible · presentación · etiqueta	No cierra el MVP

La puerta 0 es la que impide que la **velocidad del asistente se convierta en deuda**.

La trazabilidad se puede recorrer

Un evaluador puede seguir una regla de negocio hasta la prueba que la verifica:

```
handbook/03 (MVP: ≤3 reglas duras)
→ SPEC-PRD-001 · capacidad C-RUL
→ ADR-003 · Rule Engine v1 dentro del BC
→ SPEC-DOM-006 · HR-03 descanso mínimo
→ SPEC-ACC-004 · escenarios Given/When/Then
→ PBI-006 · worklogs/PBI-006-rule-engine-v1/ (3 iteraciones)
→ PR #28 · Gate 2 documentado
```

Gate 2 registrado por integración: #13 · #14 · #18+#19 · #22+#23 · #28 · #30 · #31 · #34+#35 · #36

No es una afirmación mía sobre el proceso: está en el repositorio, fechado.

Qué enseñó el experimento

- 1. El asistente rinde con **contexto escrito preciso**, no con prompts largos. Donde la spec era ambigua, el agente **inventó alcance** y hubo que retroceder.
- 2. La trazabilidad no es burocracia: permite **cambiar de modelo, de herramienta o de sesión** sin perder el proyecto.
- 3. Hay que decidir explícitamente **qué no se delega**. Aquí: aprobación de specs, validación de la demo y criterio de alcance.

Límites declarados, no escondidos

Límite	Estado
Sin pruebas E2E automáticas sobre la interfaz	Reconocido; verificación de UI es manual vía runbook
Cobertura de aceptación en capa de servicio	Deliberado para el corte
Reglas avanzadas del DOCX	Documentadas en specs, no implementadas
Compliance como módulo propio	Absorbido; requiere ADR futuro para separarse

Tres semanas, cuatro etapas

Etapa	Resultado	Hitos
Fundación	Repositorio gobernado, sin código de producto	Handbook, ADR-001...004, contratos de agente
Núcleo	Maestros y acceso	PBI-001...004, PBI-008
Planificación	Calendario, reglas duras, ausencias	PBI-005...007, PBI-009, PBI-014
Cierre	Explicación de reglas, UX, arranque en frío, etiqueta	PBI-011, PBI-013, PBI-015, PBI-010

Que la primera etapa **no produjera código de producto** fue una decisión, no un retraso.

Fuera del corte: optimización automática · IA que genera turnos · app nativa · tiempo real · informes avanzados · nube como vía de evaluación

Cierre

En desarrollo asistido por IA:

la especificación es la arquitectura ejecutable,

y la trazabilidad convierte una sesión de asistente en un sistema de ingeniería.

Producto en funcionamiento → **slides de producto** + [docs/runbook-local.md](#)

Código y evidencia → etiqueta `mvp-0.1`